PersianBits Voting System

CST2550 Group Coursework Report

Group Name: PersianBits

Team Members & Student ID

Team Leader / Scrum Master: Marjan Haghighatdoost (M01078278)

Secretary: Ronika Mohammadi Bazargani (M01046675)

Developer: Danial Nazari (M01027489)

Developer: Danial Asadi (M01074847)

Tester / Debugger / UI Designer: Mohammad Darvishi (M01031166)

Module: CST2550
Software Engineering Management and Development

Submission Date: 10 April 2026
Academic Year: 2025–2026

Contents

1. Introduction 3

2. Design 3
   2.1 Data Structure Selection and Justification 3
   2.2 Algorithm Design and Pseudo Code 4
   2.3 Time Complexity Summary 5

3. Testing 5
   3.1 Testing Approach 5
   3.2 Test Cases 6

4. Conclusion 7
   4.1 Summary of Work Done 7
   4.2 Limitations and Critical Reflection 7
   4.3 Future Improvements 7

5. References 8

1. Introduction

The PersianBits Voting System is a full-stack web application that enables organisations and institutions to create, manage, and analyse polls of five distinct types: single-candidate elections, rating-scale surveys, multi-choice questions, ranked-choice ballots using the Borda count method, and free-text responses. Built with ASP.NET Core 9 Razor Pages, Entity Framework Core 9, and SQLite, the system supports three user roles — Voter, Administrator, and Developer — each with dedicated dashboards and access controls. Voters join polls using a session code issued by the administrator, cast their ballot, and can later review their submission or view the aggregate results once a poll is closed.

The central technical challenge addressed in this project is the efficient aggregation of votes across large ballot sets. Rather than relying on standard library collections or repeated LINQ query passes, the team designed and implemented a custom open-addressing hash map (IntHashMap) and a fixed-slot vote tally engine (VoteTally) from scratch, satisfying the module requirement to build bespoke data structures. These components replaced what were previously O(n·k) aggregation loops with a single O(n) pass per election, where n is the number of votes and k is the number of candidates or options (Cormen et al., 2022).

This report is structured as follows: Section 2 presents the design decisions, data structure justification, and algorithm analysis using pseudo code. Section 3 describes the unit testing approach and presents a table of test cases. Section 4 concludes with a summary of completed work, identified limitations, and reflections on how the team would approach a similar project in future. References are provided in Section 5.

2. Design

2.1 Data Structure Selection and Justification

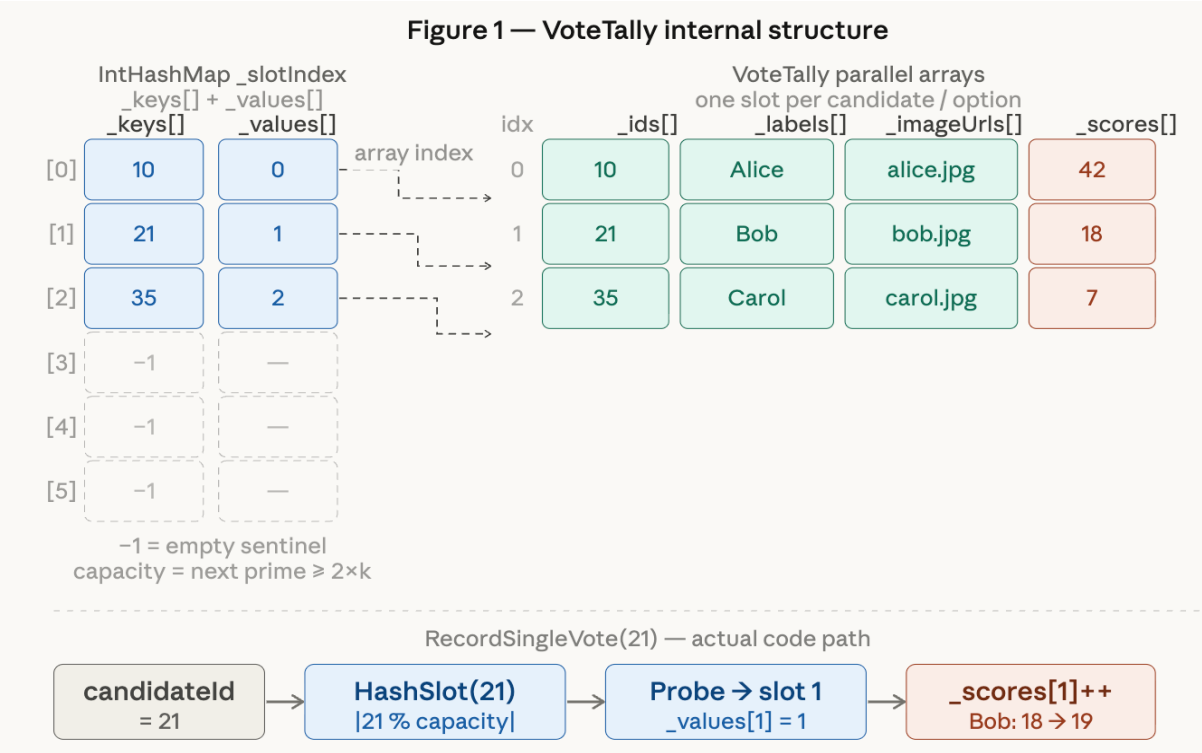
The vote aggregation task requires mapping an integer entity ID (candidate or option) to an integer vote count, performing both insert and lookup in sub-linear time. Three candidate data structures were evaluated against this requirement (Table 1).

Data Structure	Add	Lookup	Space	Justification
Array with linear scan	O(1)	O(n)	O(n)	Lookup is too slow for large candidate sets.
Binary Search Tree (BST)	O(log n)	O(log n)	O(n)	Faster than array, but node allocation reduces cache efficiency.
Hash Map — open addressing (selected)	O(1) avg	O(1) avg	O(c)	Optimal average-case; contiguous arrays give best cache locality.

Table 1: Data structure comparison for vote aggregation.

An open-addressing hash map with linear probing was selected because all data resides in two contiguous integer arrays (_keys[] and _values[]), maximising CPU cache utilisation compared to node-based structures. The capacity is set to the next prime number greater than or equal to 2 × itemCount, which keeps the load factor at most 0.5 and bounds the expected probe length to 1.5 slots on average (Knuth, 1998). A sentinel value of −1 marks empty slots safely because all entity IDs in this system are positive AUTOINCREMENT integers, so −1 cannot be a valid key.

The VoteTally class wraps IntHashMap with three parallel arrays — _ids[], _labels[], and _scores[] — one slot per candidate or option. At construction, each entity ID is mapped to its array index via IntHashMap.Add, giving O(1) average-case index resolution per vote record. The GetRankedResults method produces the final sorted output in O(k log k) time, where k is the number of candidates or options (always small in practice: typically 2–10).



2.2 Algorithm Design and Pseudo Code

The following pseudo code describes the key algorithms. All algorithms are expressed at a design level; the corresponding C# implementation is in VotingSystem.Domain/IntHashMap.cs and VotingSystem.Domain/VoteTally.cs.

Algorithm 1 — IntHashMap: HashSlot and Linear Probe

```
FUNCTION HashSlot(key) → integer:
    RETURN |key MOD capacity|
END FUNCTION

FUNCTION Probe(key) → slotIndex:
    i ← HashSlot(key)
    WHILE keys[i] ≠ EMPTY AND keys[i] ≠ key DO
        i ← (i + 1) MOD capacity
    END WHILE
    RETURN i
END FUNCTION
```

Algorithm 2 — IntHashMap: Add and TryGetValue

```
FUNCTION Add(key, value):
    slot ← Probe(key)
    keys[slot] ← key
    values[slot] ← value
END FUNCTION

FUNCTION TryGetValue(key) → (found: boolean, value: integer):
    slot ← Probe(key)
    IF keys[slot] = key THEN
        RETURN (true, values[slot])
    ELSE
        RETURN (false, 0)
    END IF
END FUNCTION
```

Algorithm 3 — VoteTally: Construction

```
FUNCTION Construct(ids[], labels[]):
    scores[] ← array of zeros, length = |ids|
    slotIndex ← new IntHashMap(capacity = |ids|)
    FOR i FROM 0 TO |ids| - 1 DO
        slotIndex.Add(ids[i], i)
    END FOR
END FUNCTION
```

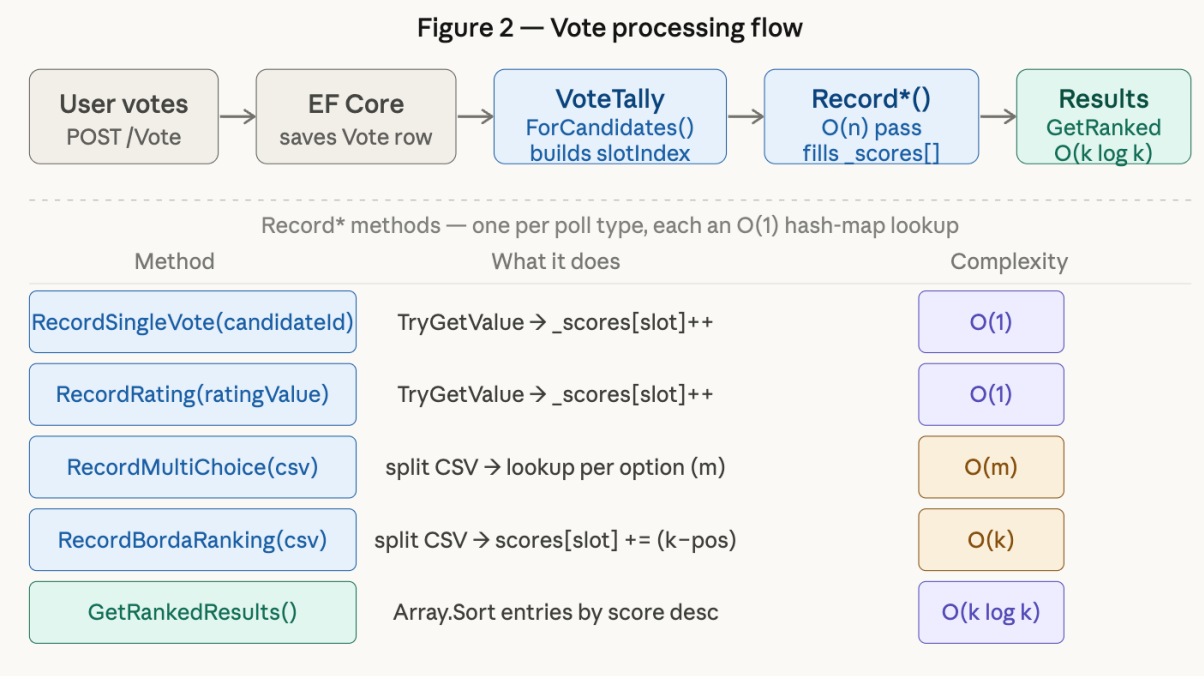
Algorithm 4 — VoteTally: Recording and Ranking Votes

```
FUNCTION RecordSingleVote(candidateId):
    IF candidateId IS NOT NULL THEN
        (found, slot) ← slotIndex.TryGetValue(candidateId)
        IF found THEN
            scores[slot] ← scores[slot] + 1
            ballotCount ← ballotCount + 1
        END IF
    END IF
END FUNCTION

FUNCTION RecordBordaRanking(rankingOrderCSV):
    IF rankingOrderCSV IS EMPTY THEN RETURN
    parts ← SPLIT rankingOrderCSV BY ","
    k ← |parts|
    FOR pos FROM 0 TO k - 1 DO
        id ← PARSE parts[pos] AS INTEGER
        (found, slot) ← slotIndex.TryGetValue(id)
        IF found THEN
            scores[slot] ← scores[slot] + (k - pos)
        END IF
    END FOR
    ballotCount ← ballotCount + 1
END FUNCTION

FUNCTION GetRankedResults() → sorted TallyEntry[]:
    FOR i FROM 0 TO |ids| - 1 DO
        pct ← IF ballotCount > 0 THEN scores[i] / ballotCount × 100 ELSE 0
        entries[i] ← (ids[i], labels[i], scores[i], pct)
    END FOR
    SORT entries BY score DESCENDING
```

```
    RETURN entries
END FUNCTION
```



2.3 Time Complexity Summary

Operation	Average Case	Worst Case	Notes
IntHashMap.Add	O(1)	O(n)	n = table capacity; worst case when all keys collide
IntHashMap.TryGetValue	O(1)	O(n)	Load factor ≤ 0.5 bounds expected probes to ≤ 1.5
VoteTally.RecordSingleVote	O(1)	O(n)	One hash-map lookup per vote
VoteTally.RecordRating	O(1)	O(n)	One hash-map lookup per vote
VoteTally.RecordMultiChoice	O(m)	O(n·m)	m = selections per ballot
VoteTally.RecordBordaRanking	O(k)	O(n·k)	k = number of ranked options
VoteTally.GetRankedResults	O(k log k)	O(k log k)	Sort of k entries; k always small in practice
VoteTally construction	O(k)	O(k)	One Add call per candidate or option

Table 2: Time complexity of all core operations.

3. Testing

3.1 Testing Approach

A unit testing strategy was adopted using the MSTest framework (Microsoft.VisualStudio.TestTools.UnitTesting). Tests target three core components of the Domain layer: IntHashMap, VoteTally, and BCryptHelper. Each test class is independent with

no database or web-layer dependencies, enabling fast, deterministic execution via dotnet test (Microsoft, 2024).

Test cases were designed using the following techniques:

- Equivalence partitioning — valid keys, missing keys, null and empty inputs divided into classes that should behave identically.
- Boundary value analysis — empty maps, single-item maps, inputs at the edges of the valid range (e.g. rating values 0 and 6 around the 1–5 scale boundary).
- Collision testing — keys deliberately chosen so that key MOD capacity = 0 for multiple keys, verifying that linear probing resolves all collisions correctly.
- Scale testing — 50-item maps with non-sequential IDs to simulate realistic database AUTOINCREMENT values.

All tests must pass before a pull request can be merged into the main branch. The full test run is executed with dotnet test from the VotingSystem.Tests project directory. No mocking frameworks are used; the Domain objects are instantiated directly.

3.2 Test Cases

Table 3 lists all 32 unit tests. TC01–TC16 appear below; TC17–TC32 continue on the following page.

ID	Component	Description	Input	Expected Output	Result
TC01	IntHashMap	Add and retrieve single item	map(5); Add(1,100); TryGet(1)	true, value = 100	Pass
TC02	IntHashMap	Retrieve missing key	map(4); Add(7,99); TryGet(42)	false	Pass
TC03	IntHashMap	Empty map returns false	map(4); TryGet(1)	false	Pass
TC04	IntHashMap	Multiple entries correct	map(5); Add(1,10),(2,20),(3,30)	All values correct	Pass
TC05	IntHashMap	Two colliding keys	keys 5 & 10 hash to slot 0	Both retrieved	Pass
TC06	IntHashMap	Chain of colliding keys	keys 7, 14, 21 hash to slot 0	All three retrieved	Pass
TC07	IntHashMap	Scale — 50 sequential items	Add keys 1–50, value = key×10	All 50 correct	Pass
TC08	IntHashMap	Non-sequential large IDs	IDs: 101, 205, 307, 412, 599	All retrieved	Pass
TC09	Election	Votes counted per candidate	2×Alice, 1×Bob	Alice=2, Bob=1, Carol=0	Pass
TC10	Election	Null candidate not counted	RecordSingleVote(null)	BallotCount = 0	Pass
TC11	Election	Unknown ID not counted	RecordSingleVote(999)	BallotCount = 0	Pass

ID	Component	Description	Input	Expected Output	Result
TC12	Election	Results sorted descending	3×Alice, 2×Bob, 1×Carol	Alice, Bob, Carol	Pass
TC13	Election	Percentages sum to 100	1 vote each, 3 candidates	Total % = 100 (±0.2)	Pass
TC14	Election	Zero votes = 0% everywhere	No votes cast	All percentages = 0	Pass
TC15	Election	BallotCount matches votes	3 vote calls	BallotCount = 3	Pass
TC16	Rating	Weighted average correct	Ratings: 5, 3	WeightedAverage = 4.0	Pass

Table 3a: Unit test cases TC01–TC16 (all passed).

ID	Component	Description	Input	Expected Output	Result
TC17	Rating	All 5-star = avg 5.0	10× RecordRating(5)	5.0	Pass
TC18	Rating	All 1-star = avg 1.0	5× RecordRating(1)	1.0	Pass
TC19	Rating	Null rating not counted	RecordRating(null)	BallotCount = 0	Pass
TC20	Rating	Out-of-range ignored	RecordRating(0) and (6)	BallotCount = 0	Pass
TC21	Multi-choice	Each option counted correctly	Voter1: 10,20; Voter2: 10,30	Pizza=2, Sushi=2, Tacos=1	Pass
TC22	Multi-choice	Ballot counted once per voter	RecordMultiChoice("10,20,30")	BallotCount = 1	Pass
TC23	Multi-choice	Null CSV ignored	RecordMultiChoice(null)	BallotCount = 0	Pass
TC24	Multi-choice	Unknown option ID ignored	RecordMultiChoice("999")	BallotCount = 0	Pass
TC25	Ranking	Borda points by position	RecordBordaRanking("1,2,3")	A=3, B=2, C=1	Pass
TC26	Ranking	Scores accumulate across voters	Voter1: 1>2>3; Voter2: 3>1>2	A=5, B=3, C=4	Pass
TC27	Ranking	Consensus winner ranked first	5× "1,2,3"	A ranked 1st	Pass
TC28	Ranking	Null input not counted	RecordBordaRanking(null)	BallotCount = 0	Pass
TC29	BCryptHelper	Hash differs from plain text	HashPassword("secret123")	Hash ≠ "secret123"	Pass
TC30	BCryptHelper	Correct password verifies	Verify("secret123", hash)	true	Pass

ID	Component	Description	Input	Expected Output	Result
TC31	BCryptHelper	Wrong password fails	Verify("wrong", hash)	false	Pass
TC32	BCryptHelper	Each hash unique (salt)	Hash("abc") twice	Two different hashes	Pass

Table 3b: Unit test cases TC17–TC32 (all passed).

4. Conclusion

4.1 Summary of Work Done

The PersianBits Voting System was delivered as a fully functional ASP.NET Core 9 web application supporting five poll types, three user roles, session-code-based voter access, live results pages, in-app notifications, and a developer administration dashboard. The project was managed using a four-sprint Scrum process with a GitHub repository, pull-request reviews, and daily standup records throughout the development period from January to April 2026.

The key algorithmic contribution is the IntHashMap and VoteTally pair. IntHashMap is a fixed-capacity open-addressing hash map with linear probing, implemented entirely from scratch without any standard library collection types. VoteTally uses IntHashMap as a slot-index over parallel arrays, reducing vote counting from O(n·k) repeated LINQ aggregations to a single O(n) pass. A comprehensive MSTest suite of 32 unit tests covers all five poll types, hash-map edge cases, and password hashing.

4.2 Limitations and Critical Reflection

- **Fixed capacity:** Fixed capacity requires the number of items to be known at construction. If a poll is extended with additional candidates, the map must be rebuilt. A dynamic resizing strategy would make the structure more robust.
- **No deletion support:** The linear-probing scheme does not support key removal without tombstone markers. This was acceptable for the read-heavy, write-once usage pattern of this system but limits reusability in other contexts.
- **Late introduction of the custom data structure:** The custom data structure was implemented in Sprint 4 rather than from the outset, requiring the replacement of functioning LINQ aggregations with the new VoteTally layer and adding unnecessary rework.
- **Third-party dependency (EF Core):** Entity Framework Core is used for all database access. Although part of the standard .NET ecosystem, it is technically a third-party library. Explicit clarification on permissibility should have been sought earlier in the project.

4.3 Future Improvements

- **Dynamic resizing:** Implement automatic capacity doubling when the load factor exceeds a threshold, with re-hashing of all existing entries.
- **Double hashing:** Replace linear probing with a secondary hash function to reduce primary clustering and improve worst-case probe lengths.
- **Earlier data-structure planning:** In a future project, design and implement the custom data structure in Sprint 1, using it as the foundation for all subsequent features rather than retrofitting it in a later sprint.
- **Expanded integration tests:** Add end-to-end tests using WebApplicationFactory to catch integration issues that unit tests alone cannot detect.

5. References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: MIT Press. [Referenced in IntHashMap.cs and VoteTally.cs for hash-table design and linear probing analysis.]

Knuth, D.E. (1998) The Art of Computer Programming, Volume 3: Sorting and Searching. 2nd edn. Reading, MA: Addison-Wesley. [Referenced in IntHashMap.cs for load factor analysis and expected probe length.]

Microsoft (2024) Unit testing C# with MSTest and .NET. Available at: https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-with-mstest (Accessed: 5 April 2026). [Referenced in VotingSystem.Tests project setup.]

Microsoft (2024) Entity Framework Core documentation. Available at: https://learn.microsoft.com/en-us/ef/core (Accessed: 5 April 2026). [Referenced in VotingSystem.Domain/AppDbContext.cs.]

SQLite Consortium (2024) SQLite documentation. Available at: https://www.sqlite.org/docs.html (Accessed: 5 April 2026). [Referenced in Database/database.sql schema file.]